

Informe del Trabajo Integrador de AyED

Integrantes: Aguirre Joaquin Mateo, Godoy Alejandro Damian y Martínez Lautaro Nicolás.

Comisión: 5

Introducción

En esta etapa del trabajo practico se desarrollaron e implementaron distintos métodos de búsqueda y procesamiento de datos con el objetivo de comparar su funcionamiento y rendimiento.

El enfoque principal estuvo puesto en la utilización de estructuras de datos eficientes para optimizar las búsquedas dentro de una colección de información.

Explicacion del método BuscarConOtro

Primero se utiliza el método ContarOcuurrencias, cuya función es recorrer todos los datos recibidos en la lista y contar cuantas veces aparece cada palabra.

Los datos llegan como una List<string>, por lo que fue necesario transformarlos en objetos de tipo Dato.

El método recorre toda la lista y verifica si el dato ya existe:

- Si ya existe, incrementa el valor de ocurrencia
- Si no existe, crea un nuevo objeto ConOcurrencia inicial igual a 1

De esta manera queda una lista donde cada palabra aparece una sola vez junto con la cantidad de veces que fue encontrada.

Luego se implementó el método BuscarConOtro, utilizando el algoritmo de ordenamiento Selection Sort.

El primer for recorre toda la lista y, en cada posición, busca cual es el elemento con mayor cantidad de ocurrencias comparando uno por uno mediante el segundo for.

Cuando encuentra el elemento con mayor ocurrencia, guarda su posición en la variable Max.

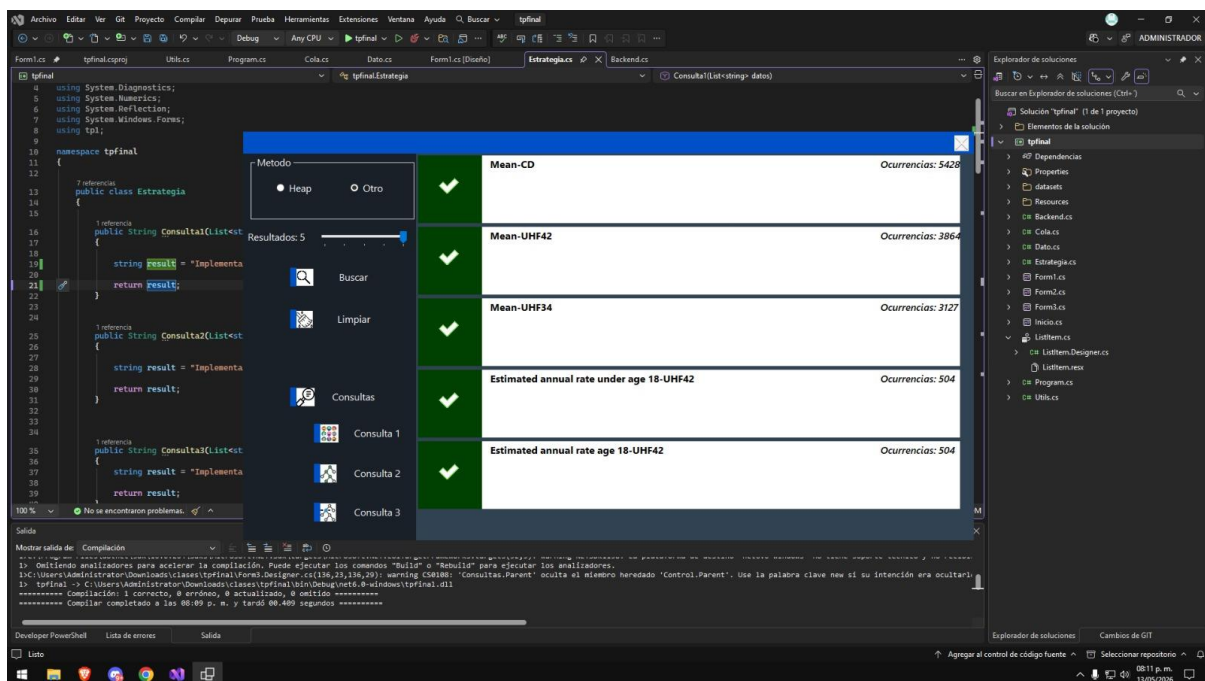
Al finalizar la búsqueda, se realiza un intercambio utilizando una variable auxiliar (aux):

El elemento con mayor ocurrencia pasa a la posición actual y el elemento que estaba anteriormente en esa posición pasa a la posición del máximo encontrado.

De esta forma la lista queda ordenada de mayor a menor según la cantidad de ocurrencias.

Finalmente, se agregan a collected únicamente los primeros elementos pedidos por parámetro mediante la variable cantidad.

Visualización del método BuscarConOtro



Explicación del método BuscarConHeap

Para implementar la búsqueda utilizando Heap, primero se utilizó el método ContarOcurrencias, ya que era necesario obtener una lista de objetos Dato con su cantidad de apariciones.

Después se implementó una estructura Max heap, donde el elemento con mayor cantidad de ocurrencias siempre queda ubicado en la raíz.

Para mantener correctamente la estructura se crearon los métodos auxiliares ReordenarHeap y ConstruirHeap.

Método ReordenarHeap

ReordenarHeap es el encargado de mantener el orden dentro del heap.

El método revisa un nodo y compara su ocurrencia con la de sus hijos izquierdo y derecho.

Si alguno de los hijos posee una ocurrencia mayor:

Primero se intercambian las posiciones pasando el mayor arriba y se vuelve a ejecutar ReordenarHeap recursivamente sobre la nueva posición.

Esto permite conservar la propiedad de Max Heap, donde el valor más grande siempre permanece arriba.

Método ConstruirHeap

El método ConstruirHeap recorre la lista desde la mitad hacia el inicio y aplica ReordenarHeap en cada posición.

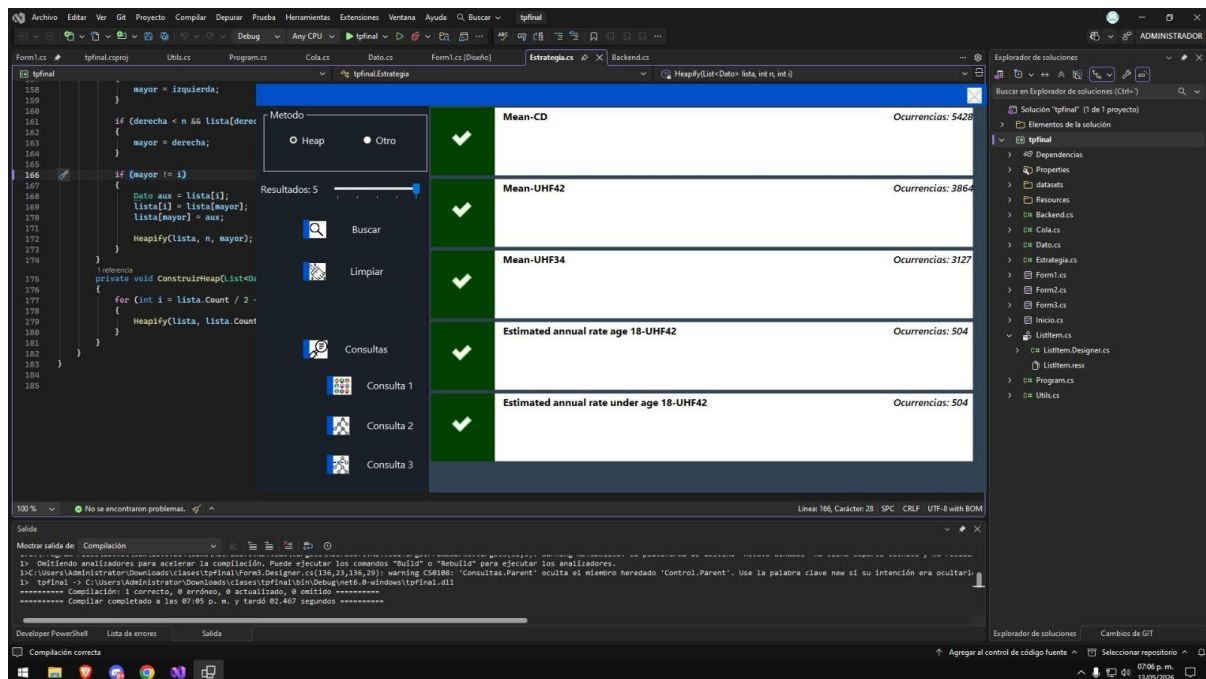
El objetivo es transformar la lista completa en una estructura Heap válida.

Funcionamiento de BuscarConHeap

- El método primero cuenta las ocurrencias de cada dato.
- Una vez armado el Heap.
- El elemento con mayor ocurrencia queda en la posición 0.
- Ese elemento se agrega a collected.
- La raíz se reemplaza por el último elemento del Heap
- El tamaño lógico disminuye.
- Se vuelve a ejecutar ReordenarHeap para reordenar la estructura

Este proceso se repite hasta obtener la cantidad de elementos solicitados.

Visualización del método BuscarConHeap



Explicación de la Consulta 1

La Consulta 1 tiene como objetivo comparar el rendimiento de los dos métodos de búsqueda implementados anteriormente: BuscarConHeap y BuscarConOtro.

Para realizar la comparación se utiliza la clase Stopwatch, que permite medir el tiempo de ejecución de un algoritmo.

Primero se crean dos listas auxiliares:

- heap, utilizada para almacenar los resultados obtenidos mediante BuscarConHeap.
- ordenados, utilizada para almacenar los resultados obtenidos mediante BuscarConOtro.

Luego se define una variable llamada repeticiones con valor 100. Esto permite ejecutar cada algoritmo varias veces para obtener una medición más representativa y reducir posibles variaciones de tiempo.

El cronómetro se inicia utilizando Restart() y se ejecuta BuscarConHeap cien veces consecutivas solicitando los cinco elementos con mayor cantidad de ocurrencias.

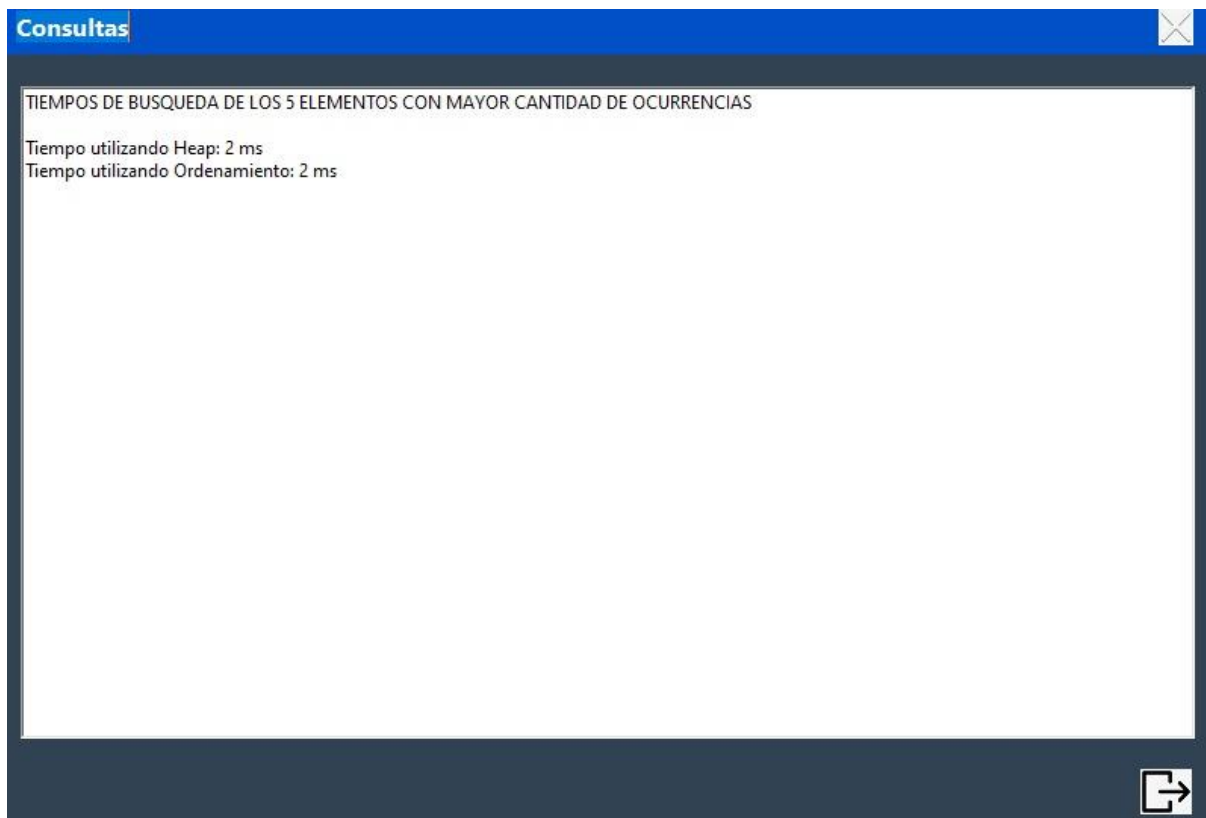
Al finalizar las ejecuciones se detiene el cronómetro mediante Stop() y se almacena el tiempo total transcurrido en la variable tiempoHeap.

Posteriormente se repite exactamente el mismo procedimiento utilizando `BuscarConOtro`, almacenando el resultado en `tiempoOrden`.

Finalmente, el método devuelve una cadena de texto mostrando ambos tiempos de ejecución en milisegundos.

El objetivo principal de esta consulta es analizar cuál de las dos estrategias resulta más eficiente para obtener los elementos con mayor cantidad de ocurrencias dentro de una colección de datos.

Visualización consulta 1



Explicación de la Consulta 2

La Consulta 2 tiene como finalidad mostrar el recorrido de la rama izquierda de la estructura Heap.

Primero se utiliza el método ContarOcuurrencias para generar una lista de objetos Dato junto con la cantidad de veces que aparece cada uno.

Posteriormente se ejecuta ConstruirHeap para transformar la lista en una Max Heap válida.

Una vez construida la Heap, se comienza el recorrido desde la posición 0, correspondiente a la raíz de la estructura.

En cada iteración se agrega al resultado el texto almacenado en el nodo actual y se calcula la posición de su hijo izquierdo mediante la fórmula:

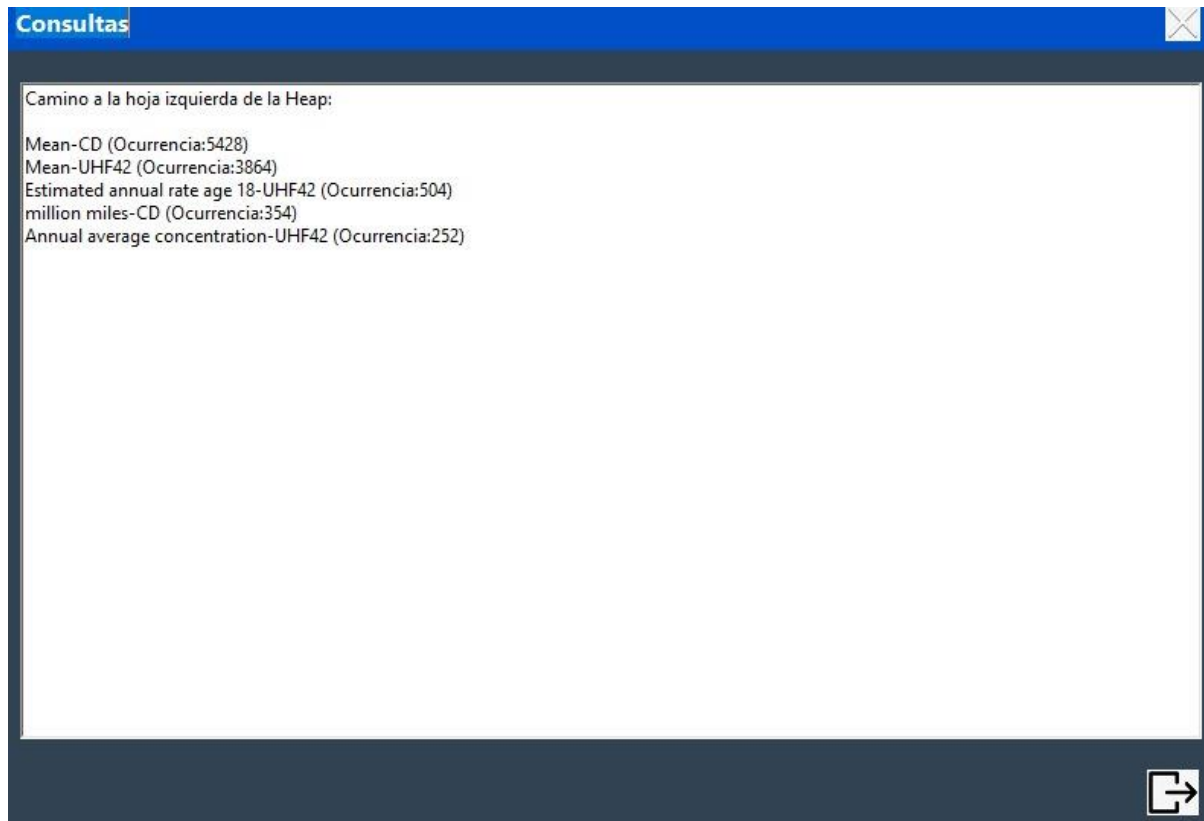
$$\text{hijolzq} = 2 * \text{actual} + 1$$

Si el hijo izquierdo existe dentro de la lista, se agrega una flecha para indicar la continuidad del recorrido.

Luego la variable actual toma el valor de hijolzq y el proceso continúa hasta llegar a un nodo que ya no posea hijo izquierdo.

Como resultado se obtiene una representación textual de la rama izquierda de la Heap, comenzando desde la raíz y descendiendo nivel por nivel únicamente por los hijos izquierdos.

Visualización de la consulta 2



Explicación de la Consulta 3

La Consulta 3 tiene como objetivo mostrar todos los nodos de la Heap organizados por niveles.

Inicialmente se utiliza el método `ContarOurrencias` para obtener la lista de datos junto con sus respectivas cantidades de ocurrencias.

Luego se ejecuta `ConstruirHeap` para transformar dicha lista en una Max Heap válida.

Una vez construida la estructura, se realiza un recorrido por niveles utilizando variables auxiliares que permiten identificar qué elementos pertenecen a cada nivel del árbol.

La variable `nivel` almacena el número de nivel actual.

La variable `i` indica la posición desde donde comienza cada nivel dentro de la lista.

La variable `elementosNivel` determina la cantidad máxima de nodos que puede contener cada nivel.

El recorrido comienza con un único elemento en el nivel 0, correspondiente a la raíz.

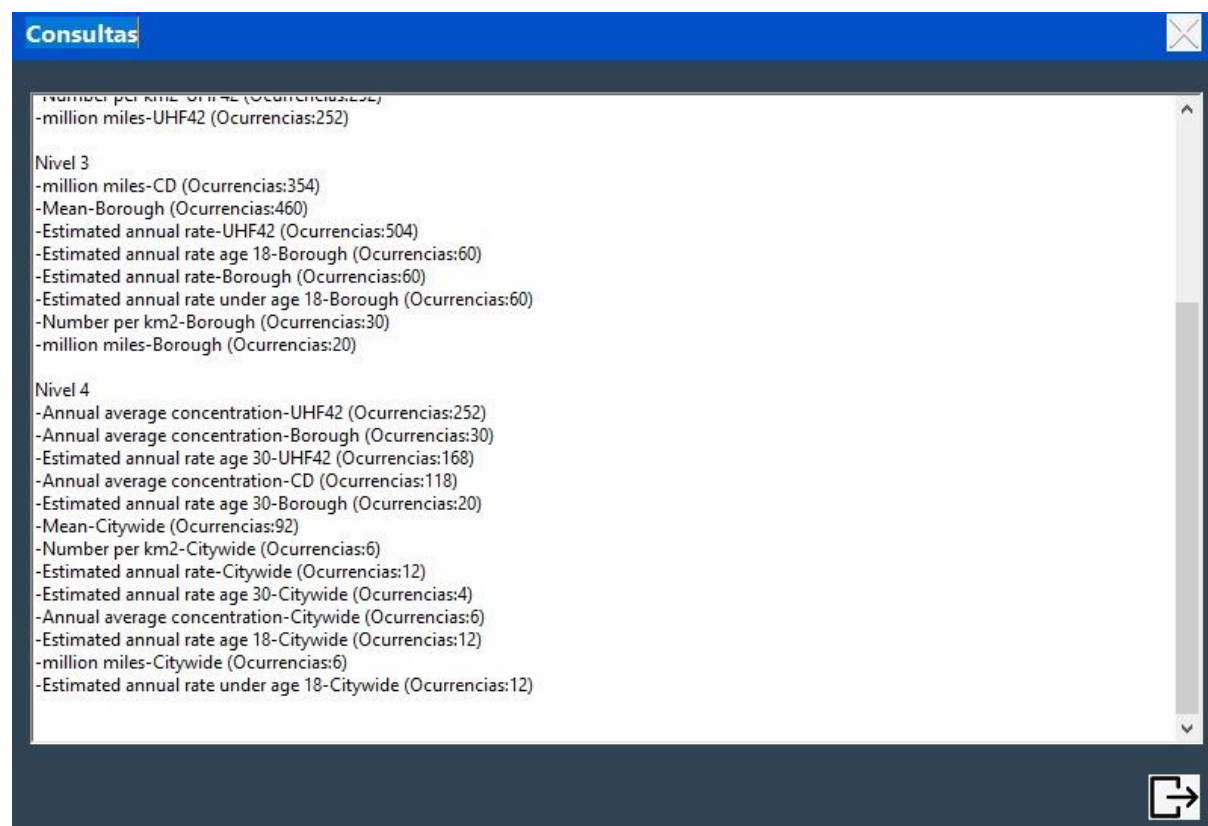
Posteriormente, la cantidad de elementos posibles se duplica en cada nivel siguiendo la propiedad de los árboles binarios completos:

- Nivel 0: hasta 1 nodo.
- Nivel 1: hasta 2 nodos.
- Nivel 2: hasta 4 nodos.
- Nivel 3: hasta 8 nodos.

Para cada nivel se muestran el texto almacenado y su cantidad de ocurrencias.

Finalmente, el método devuelve una representación organizada de toda la Heap, permitiendo visualizar claramente cómo se distribuyen los nodos dentro de la estructura y comprobar que se mantiene la propiedad de Max Heap, donde los elementos con mayor cantidad de ocurrencias se encuentran en los niveles superiores.

Visualización de la consulta 3





-Number per km2-UHF42 (Ocurrencias:252)
-million miles-UHF42 (Ocurrencias:252)

Nivel 3

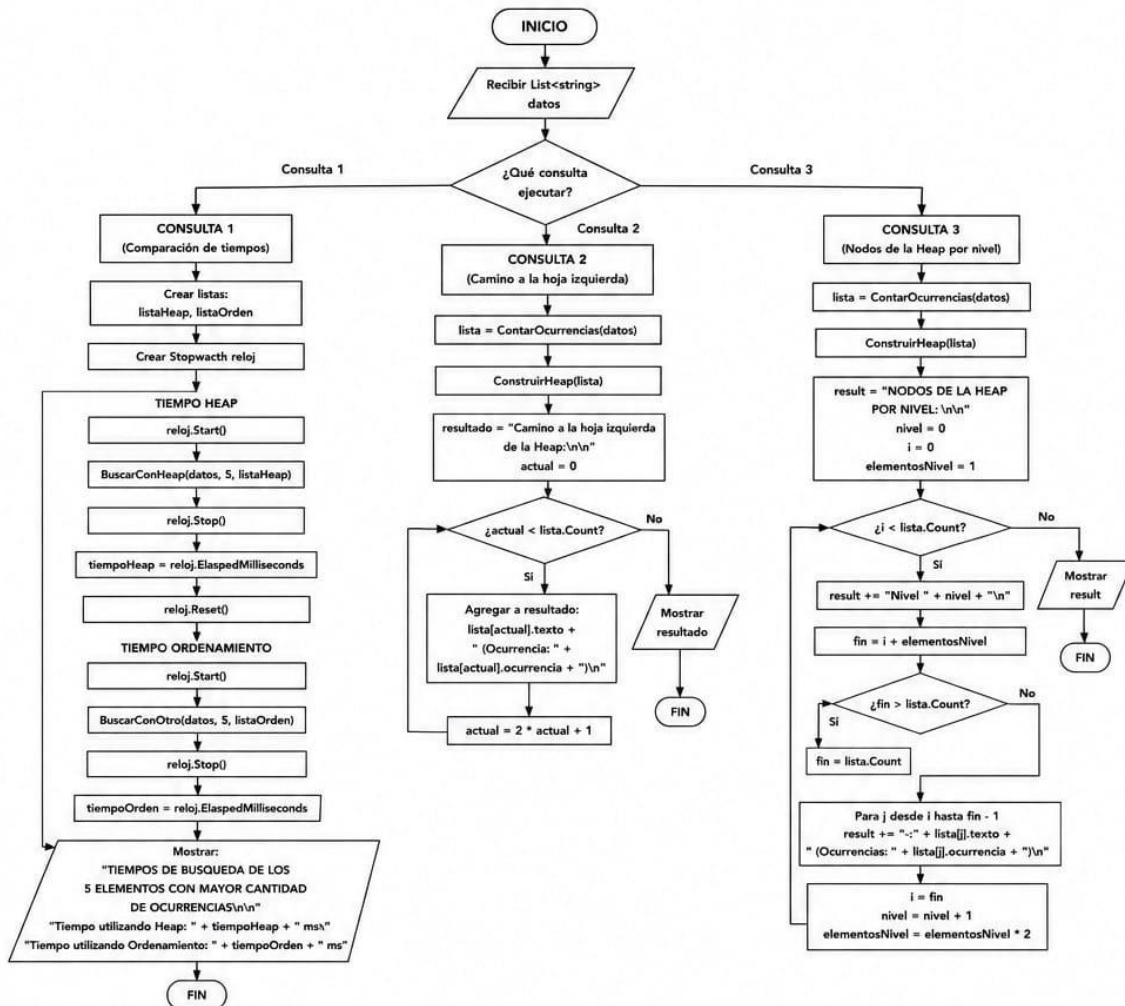
-million miles-CD (Ocurrencias:354)
-Mean-Borough (Ocurrencias:460)
-Estimated annual rate-UHF42 (Ocurrencias:504)
-Estimated annual rate age 18-Borough (Ocurrencias:60)
-Estimated annual rate-Borough (Ocurrencias:60)
-Estimated annual rate under age 18-Borough (Ocurrencias:60)
-Number per km2-Borough (Ocurrencias:30)
-million miles-Borough (Ocurrencias:20)

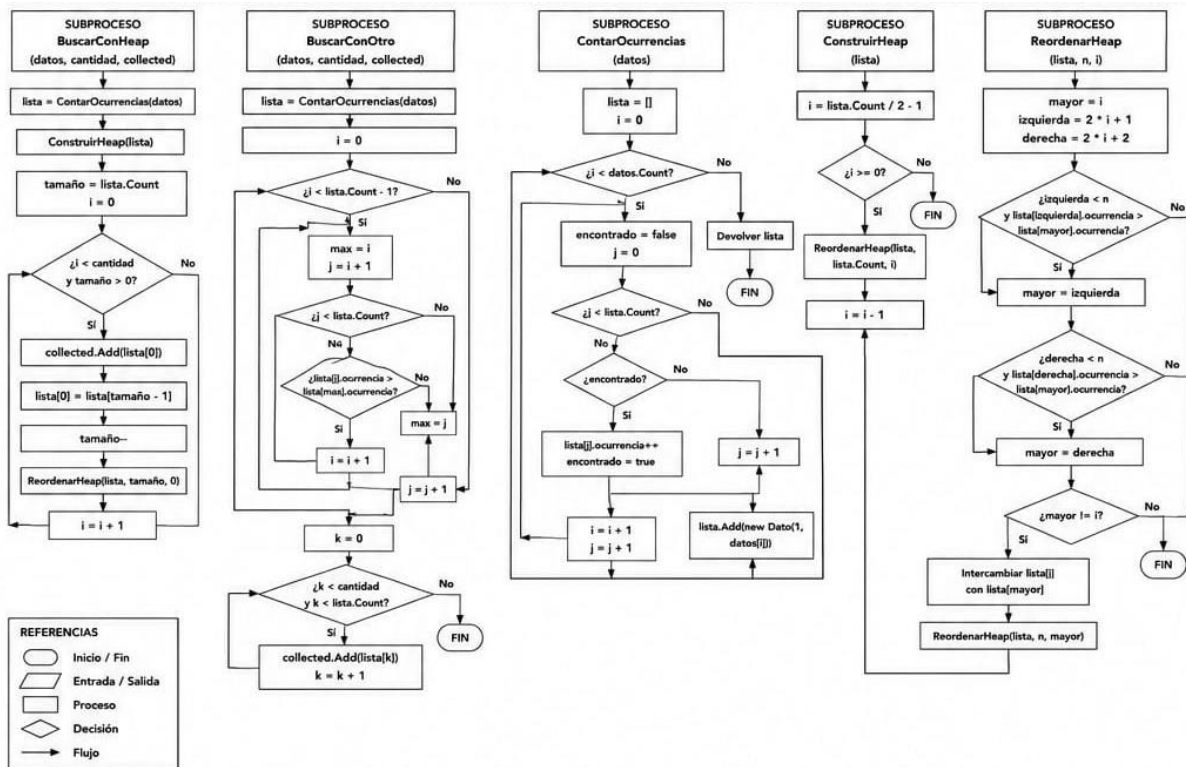
Nivel 4

-Annual average concentration-UHF42 (Ocurrencias:252)
-Annual average concentration-Borough (Ocurrencias:30)
-Estimated annual rate age 30-UHF42 (Ocurrencias:168)
-Annual average concentration-CD (Ocurrencias:118)
-Estimated annual rate age 30-Borough (Ocurrencias:20)
-Mean-Citywide (Ocurrencias:92)
-Number per km2-Citywide (Ocurrencias:6)
-Estimated annual rate-Citywide (Ocurrencias:12)
-Estimated annual rate age 30-Citywide (Ocurrencias:4)
-Annual average concentration-Citywide (Ocurrencias:6)
-Estimated annual rate age 18-Citywide (Ocurrencias:12)
-million miles-Citywide (Ocurrencias:6)
-Estimated annual rate under age 18-Citywide (Ocurrencias:12)



Diagramas de flujo





Detalles de implementación

Durante el desarrollo del trabajo implementamos dos formas distintas de obtener los elementos con mayor cantidad de ocurrencias: una utilizando una Heap y otra mediante un método de ordenamiento (Selection Sort). Además, realizamos las consultas solicitadas para comparar tiempos de ejecución, recorrer la rama izquierda de la Heap y mostrar los datos organizados por niveles.

Problemas encontrados

Uno de los puntos que más nos costó fue comprender el funcionamiento de la Heap y cómo se reorganizaban los elementos para mantener la propiedad de máximo. También tuvimos algunas dudas al momento de implementar las consultas, especialmente la Consulta 1, ya que debíamos comparar los tiempos de ejecución de ambos métodos y encontrar una forma adecuada de medirlos.

Otro tema que surgió durante el desarrollo fue el uso de listas para representar la estructura de datos. Más adelante se nos sugirió que una alternativa podía ser utilizar arreglos, ya que es una implementación más clásica para las Heaps. Sin embargo, como el trabajo ya estaba avanzado y funcionando correctamente,

decidimos mantener la implementación original para evitar realizar cambios importantes a último momento que pudieran generar errores o afectar otras partes del programa.

Además, fue necesario realizar varias pruebas y correcciones para asegurarnos de que las consultas devolvieran exactamente la información solicitada por las consignas.

Posibles mejoras

Como posible mejora del trabajo, nos hubiera gustado implementar la Heap utilizando arreglos en lugar de listas, ya que es una representación más clásica de esta estructura de datos y se ajusta mejor a los conceptos vistos durante la cursada. Sin embargo, cuando surgió esta posibilidad gran parte del trabajo ya se encontraba implementada y funcionando correctamente, por lo que decidimos no realizar cambios importantes a último momento para evitar introducir errores.

También podría mejorarse la comparación de rendimiento entre los métodos implementados. Debido al tamaño del conjunto de datos utilizado, las diferencias de tiempo obtenidas fueron muy pequeñas. En una versión más avanzada sería interesante trabajar con volúmenes de datos mayores para analizar con más claridad las ventajas y desventajas de cada solución.

Conclusión

La realización de este trabajo nos permitió aplicar en la práctica varios de los temas vistos durante la cursada, especialmente los relacionados con métodos de ordenamiento, estructuras de datos y Heaps. A lo largo del desarrollo tuvimos que analizar distintas alternativas, resolver problemas de implementación y realizar varias pruebas hasta obtener los resultados esperados.

Además de aprender aspectos técnicos, el trabajo también nos ayudó a comprender la importancia de planificar una solución, verificar que los resultados sean correctos y trabajar en equipo para resolver las dificultades que fueron apareciendo. En general, consideramos que fue una experiencia positiva porque nos permitió afianzar los conocimientos adquiridos durante la materia y entender mejor cómo aplicarlos en un problema real.